COMPLETE REFERENCE

DATA STRUCTURES & ALGORITHMS

# DSA Master Handbook

Every core topic explained — with theory, step-by-step algorithms, clean Python code, complexity analysis, and mock viva questions with model answers.

---

Coverage: **Complexity · Arrays · Strings · Linked Lists · Stacks · Queues · Hashing**
**Recursion · Trees · Heaps · Tries · Graphs · Sorting · Searching**
**Divide & Conquer · Greedy · Backtracking · Dynamic Programming**

Language: Python 3 — every algorithm includes runnable code and interview-style Q&A.

# Table of Contents

# 1. Complexity Analysis

*Every DSA interview begins here. You must be able to derive and justify the time/space complexity of any code you write.*

## 1.1 Asymptotic notation

- **Big-O (O)** — upper bound / worst case ("no worse than").
- **Big-Omega (Ω)** — lower bound / best case ("at least").
- **Big-Theta (Θ)** — tight bound (upper = lower, exact growth).

## 1.2 Common growth rates (best → worst)

| Notation | Name | Example |
|---|---|---|
| O(1) | Constant | Array index access, hash lookup |
| O(log n) | Logarithmic | Binary search, balanced BST search |
| O(n) | Linear | Single loop over n items |
| O(n log n) | Linearithmic | Merge sort, heap sort, quicksort avg |
| $O(n^2)$ | Quadratic | Nested loops, bubble/insertion sort |
| $O(2^n)$ | Exponential | Naive recursion (subsets, Fibonacci) |
| O(n!) | Factorial | Permutations, brute-force TSP |

## 1.3 How to analyze

- **Drop constants:** O(2n) → O(n).
- **Drop lower-order terms:** $O(n^2 + n) \rightarrow O(n^2)$.
- **Sequential blocks add:** O(a) + O(b).
- **Nested loops multiply:** O(a × b).
- **Space complexity** counts extra memory (variables, recursion stack, auxiliary structures) — the call stack of recursion depth d is O(d).

> **Amortized analysis:** some operations are occasionally expensive but cheap on average. Appending to a dynamic array (Python list) is amortized O(1) even though an occasional resize is O(n).

## Viva — Complexity

**Q Difference between Big-O, Big-Omega and Big-Theta?**

**A** Big-O is the worst-case upper bound, Big-Omega the best-case lower bound, and Big-Theta the tight bound when upper and lower bounds match. In interviews "complexity" usually means Big-O (worst case).

**Q What's the time complexity of two separate loops over n vs two nested loops?**

**A** Separate loops add: $O(n) + O(n) = O(n)$. Nested loops multiply: $O(n) \times O(n) = O(n^2)$.

**Q What is amortized complexity? Give an example.**

**A** The average cost per operation over a sequence, even if individual ops vary. Appending to a dynamic array is amortized O(1): most appends are O(1), and the rare O(n) resize is spread across many cheap appends.

**Q Does recursion cost space?**

**A** Yes — each recursive call adds a stack frame, so recursion depth d uses O(d) auxiliary space even if no other structures are allocated.

**Q Is O(n log n) better than $O(n^2)$? Always?**

**A** Asymptotically yes, for large n. For very small n the constant factors can make an $O(n^2)$ algorithm faster, which is why hybrid sorts (Timsort) use insertion sort on tiny sub-arrays.

# 2. Arrays & Two-Pointer / Sliding Window

*Arrays are contiguous, index-accessible collections — the most fundamental structure and the basis of most interview problems.*

## 2.1 Characteristics

| Operation | Complexity | Why |
|---|---|---|
| Access by index | O(1) | Direct address arithmetic |
| Search (unsorted) | O(n) | Must scan |
| Insert/delete at end | O(1) amortized | Dynamic array append |
| Insert/delete at middle | O(n) | Shift elements |

## 2.2 Two-pointer technique

Use two indices moving toward/with each other to solve pair/subarray problems in O(n) instead of O(n²).

```python
def two_sum_sorted(arr, target):
    """Find a pair summing to target in a SORTED array. O(n) time, O(1) space."""
    left, right = 0, len(arr) - 1
    while left < right:
        s = arr[left] + arr[right]
        if s == target:
            return (left, right)
        elif s < target:
            left += 1          # need a bigger sum
        else:
            right -= 1         # need a smaller sum
    return None
```

## 2.3 Sliding window

Maintain a window (subarray) that expands/contracts to track a running property — ideal for "max/min subarray of size k" or "longest substring" problems.

```python
def max_subarray_sum_k(arr, k):
    """Max sum of any contiguous subarray of size k. O(n)."""
    window = sum(arr[:k])
    best = window
    for i in range(k, len(arr)):
        window += arr[i] - arr[i - k]   # slide: add new, drop oldest
        best = max(best, window)
    return best
```

**Kadane's algorithm** (max subarray sum, any size) is a classic — track the best sum ending at each index: `cur = max(x, cur + x); best = max(best, cur)` in O(n).

## Viva — Arrays

**Q Why is array access O(1)?**

**A** Elements are stored contiguously, so the address of index i is base + i×size — a constant-time arithmetic calculation, no scanning.

**Q When does the two-pointer technique require a sorted array?**

**A** For pair-sum style problems where moving a pointer must predictably increase or decrease the sum. For window problems on positive numbers, sorting isn't needed.

**Q Explain Kadane's algorithm.**

**A** It finds the maximum-sum contiguous subarray in O(n) by keeping the best sum ending at the current index (reset to the current element if the running sum goes negative) and tracking the global best.

**Q Array vs linked list — when do you pick each?**

**A** Array for fast random access and cache-friendly iteration; linked list for frequent insertions/deletions in the middle without shifting, when you don't need indexed access.

# 3. Strings

*Strings are sequences of characters; in Python they are immutable, which shapes how you manipulate them.*

## 3.1 Key ideas

- Immutable → each modification creates a new string; build with a list then `"".join()` for O(n) instead of O(n²).
- Common patterns: reversal, palindrome check, anagram detection, frequency counting, substring search.

```python
def is_palindrome(s):
    """O(n) time, O(1) extra space using two pointers."""
    l, r = 0, len(s) - 1
    while l < r:
        if s[l] != s[r]:
            return False
        l += 1; r -= 1
    return True

def are_anagrams(a, b):
    """O(n) using frequency count."""
    from collections import Counter
    return Counter(a) == Counter(b)
```

## 3.2 Pattern matching — KMP idea

Naive substring search is O(n·m). **KMP (Knuth–Morris–Pratt)** pre-computes a "failure/LPS" table so it never re-checks characters, giving O(n+m). The LPS array stores, for each position, the length of the longest proper prefix that is also a suffix.

### Viva — Strings

**Q Why is repeatedly concatenating strings in a loop slow in Python?**

**A** Strings are immutable, so each `s += x` creates a new string and copies the old contents — O(n²) overall. Collecting parts in a list and calling "".join() is O(n).

**Q How do you check two strings are anagrams?**

**A** Compare character frequency counts (Counter) in O(n), or sort both and compare in O(n log n). The frequency approach is faster.

**Q What does KMP improve over naive search?**

**A** It avoids re-scanning already matched characters using a precomputed prefix (LPS) table, reducing worst-case substring search from O(n·m) to O(n+m).

# 4. Linked Lists

*A linked list stores elements in nodes, each pointing to the next — no contiguous memory, so insertion/deletion is cheap but random access is not.*

## 4.1 Types

- **Singly** — each node points to next only.
- **Doubly** — nodes have next and prev pointers (bidirectional traversal).
- **Circular** — last node links back to head.

## 4.2 Singly linked list with core operations

```
class Node:
    def __init__(self, val):
        self.val = val
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    def push_front(self, val):          # O(1)
        n = Node(val); n.next = self.head; self.head = n

    def append(self, val):              # O(n)
        n = Node(val)
        if not self.head:
            self.head = n; return
        cur = self.head
        while cur.next:
            cur = cur.next
        cur.next = n

    def reverse(self):                  # O(n) time, O(1) space
        prev, cur = None, self.head
        while cur:
            nxt = cur.next
            cur.next = prev
            prev = cur
            cur = nxt
        self.head = prev
```

## 4.3 Floyd's cycle detection (tortoise & hare)

```python
def has_cycle(head):
    """Detect a loop in O(n) time, O(1) space."""
    slow = fast = head
    while fast and fast.next:
        slow = slow.next            # 1 step
        fast = fast.next.next       # 2 steps
        if slow is fast:            # they meet inside the loop
            return True
    return False
```

### Viva — Linked Lists

**Q Linked list vs array — main trade-off?**

**A** Linked lists give O(1) insert/delete at a known node without shifting, but O(n) access and worse cache locality. Arrays give O(1) random access but O(n) middle insertion.

**Q How do you reverse a singly linked list?**

**A** Iterate with three pointers (prev, cur, next), reversing each node's next pointer, in O(n) time and O(1) space; return prev as the new head.

**Q How do you detect a cycle?**

**A** Floyd's tortoise-and-hare: advance one pointer by 1 and another by 2; if they ever meet there's a cycle. It's O(n) time and O(1) space.

**Q How do you find the middle of a linked list in one pass?**

**A** Use slow/fast pointers — when fast reaches the end, slow is at the middle.

**Q Why is a doubly linked list useful?**

**A** It allows O(1) deletion of a known node (you have prev) and backward traversal — the backbone of LRU caches and deques.

# 5. Stacks

*A stack is LIFO (Last In, First Out) — think a stack of plates. Core ops: push, pop, peek, all O(1).*

```
class Stack:
    def __init__(self):
        self._data = []
    def push(self, x): self._data.append(x)     # O(1)
    def pop(self):     return self._data.pop()   # O(1)
    def peek(self):    return self._data[-1]
    def is_empty(self):return not self._data

def is_balanced(s):
    """Check balanced brackets - classic stack problem."""
    pairs = {')': '(', ']': '[', '}': '{'}
    stack = []
    for ch in s:
        if ch in "([{":
            stack.append(ch)
        elif ch in pairs:
            if not stack or stack.pop() != pairs[ch]:
                return False
    return not stack
```

**Uses:** function call stack, undo/redo, expression evaluation (infix→postfix), backtracking, DFS, browser history.

## Viva — Stacks

**Q Give three real uses of a stack.**

**A** The function call stack (recursion), undo/redo in editors, and expression parsing/evaluation (balanced brackets, infix-to-postfix). DFS also uses a stack.

**Q How do you evaluate/validate expressions with a stack?**

**A** Push opening symbols; on a closing symbol, pop and check it matches. If the stack is empty when a match is needed or non-empty at the end, the expression is unbalanced.

**Q Can you implement a queue using two stacks?**

**A** Yes — use an "in" stack for enqueue and an "out" stack for dequeue; when out is empty, pop everything from in into out, reversing order to achieve FIFO. Amortized O(1).

# 6. Queues, Deques & Priority Queues

*A queue is FIFO (First In, First Out) — like a line at a counter.*

## 6.1 Types

- **Simple queue** — enqueue at rear, dequeue at front.
- **Circular queue** — reuses freed front space with modular indexing.
- **Deque** — double-ended; insert/remove at both ends (O(1)).
- **Priority queue** — elements served by priority, not arrival (usually a heap).

```python
from collections import deque

q = deque()
q.append(1); q.append(2)      # enqueue -> rear
front = q.popleft()          # dequeue -> front (O(1))

# Deque as both stack and queue
dq = deque()
dq.appendleft(0); dq.append(3)   # both ends O(1)
```

**Don't** use a plain Python list as a queue: `list.pop(0)` is O(n) because it shifts every element. Use `collections.deque` for O(1) both ends.

### Viva — Queues

**Q Stack vs queue?**

**A** Stack is LIFO (last in, first out); queue is FIFO (first in, first out). Stacks suit backtracking/DFS; queues suit scheduling/BFS.

**Q Why prefer deque over a list for a queue in Python?**

**A** deque gives O(1) appends and pops at both ends; a list's pop(0) is O(n) because it shifts all remaining elements.

**Q What is a priority queue and how is it implemented?**

**A** A queue where the highest/lowest priority element is dequeued first, typically implemented with a binary heap giving O(log n) insert and extract. Python's heapq provides this.

**Q Where are queues used in algorithms?**

**A** BFS traversal, level-order tree traversal, CPU/task scheduling, and buffering (producer–consumer).

# 7. Recursion & Backtracking

*Recursion solves a problem by solving smaller instances of itself. Backtracking is recursion that explores choices and undoes them.*

## 7.1 Anatomy of recursion

- **Base case** — stops the recursion (prevents infinite loops / stack overflow).
- **Recursive case** — reduces toward the base case.

```
def factorial(n):
    if n <= 1:            # base case
        return 1
    return n * factorial(n - 1)   # recursive case

def fib(n, memo={}):     # memoized: O(n) instead of O(2^n)
    if n < 2:
        return n
    if n not in memo:
        memo[n] = fib(n-1, memo) + fib(n-2, memo)
    return memo[n]
```

## 7.2 Backtracking template

Choose → explore → un-choose. Used for permutations, subsets, N-Queens, Sudoku, maze paths.

```
def permutations(nums):
    result, path, used = [], [], [False]*len(nums)
    def backtrack():
        if len(path) == len(nums):
            result.append(path[:])       # a complete solution
            return
        for i in range(len(nums)):
            if used[i]:
                continue
            used[i] = True; path.append(nums[i])   # choose
            backtrack()                            # explore
            used[i] = False; path.pop()            # un-choose
    backtrack()
    return result
```

### Viva — Recursion / Backtracking

**Q What two things must every recursive function have?**

**A** A base case that terminates, and a recursive case that moves toward the base case. Missing/incorrect base cases cause infinite recursion and stack overflow.

**Q Recursion vs iteration — trade-offs?**

**A** Recursion is often cleaner for tree/graph and divide-and-conquer problems but uses call-stack memory (O(depth)) and risks overflow. Iteration is more memory-efficient. Any recursion can be rewritten iteratively (often with an explicit stack).

**Q What is memoization and why use it?**

**A** Caching results of subproblems so they aren't recomputed. It turns exponential recursion (like naive Fibonacci, O(2^n)) into linear O(n) — it's top-down dynamic programming.

**Q Explain the backtracking pattern.**

**A** At each step make a choice, recurse to explore, then undo the choice before trying the next — systematically searching the solution space while pruning invalid branches early.

**Q What is tail recursion?**

**A** Recursion where the recursive call is the last operation, allowing some languages to optimize it into a loop (no growing stack). Python does not do tail-call optimization.

# 8. Hashing & Hash Tables

*A hash table maps keys to values using a hash function, giving average O(1) insert/search/delete — the workhorse of fast lookups.*

## 8.1 How it works

- A **hash function** converts a key to an array index.
- **Collisions** (two keys → same index) are resolved by **chaining** (linked list per bucket) or **open addressing** (probe for the next free slot).
- **Load factor** = entries / buckets; when it grows too high the table **resizes/rehashes**.

```python
# Python dict/set ARE hash tables. Common interview uses:
def two_sum(arr, target):
    """Unsorted array, O(n) using a hash map of value->index."""
    seen = {}
    for i, x in enumerate(arr):
        if target - x in seen:
            return (seen[target - x], i)
        seen[x] = i
    return None

def first_non_repeating(s):
    from collections import Counter
    counts = Counter(s)              # O(n)
    for ch in s:
        if counts[ch] == 1:
            return ch
    return None
```

### Viva — Hashing

**Q Why is hash table lookup average O(1) but worst case O(n)?**

**A** With a good hash function keys spread evenly, so lookups touch a tiny bucket — O(1). If many keys collide into one bucket (bad hash or adversarial input), the bucket degrades to a list scanned in O(n).

**Q What is a collision and how is it handled?**

**A** Two keys hashing to the same index. Handled by chaining (store a list at each bucket) or open addressing (linear/quadratic probing or double hashing to find another slot).

**Q What is the load factor?**

**A** The ratio of stored entries to buckets. When it exceeds a threshold (e.g., 0.7), the table resizes and rehashes to keep operations near O(1).

**Q What makes a good hash function?**

**A** It should be fast, deterministic, and distribute keys uniformly to minimize collisions; small input changes should scatter the output.

**Q When would you use a hash set vs hash map?**

**A** A set for membership/uniqueness checks (no values), a map/dict when you need to associate a value with each key (counts, indices, cached results).

# 9. Trees & Binary Search Trees

*A tree is a hierarchical structure of nodes with one root and no cycles. Binary trees have at most two children per node.*

## 9.1 Terminology

**Root** (top), **leaf** (no children), **height** (longest root-to-leaf edge count), **depth** (edges from root to a node), **subtree**, **balanced** (heights differ by ≤1).

## 9.2 Traversals

| Traversal | Order | Use |
|---|---|---|
| Inorder | Left, Root, Right | BST → sorted output |
| Preorder | Root, Left, Right | Copy/serialize a tree |
| Postorder | Left, Right, Root | Delete tree, evaluate expression |
| Level-order (BFS) | Level by level | Shortest path, level grouping |

```python
class TreeNode:
    def __init__(self, val):
        self.val = val; self.left = None; self.right = None

def inorder(node, out):
    if node:
        inorder(node.left, out)
        out.append(node.val)
        inorder(node.right, out)

def level_order(root):
    from collections import deque
    res, q = [], deque([root] if root else [])
    while q:
        level = []
        for _ in range(len(q)):
            n = q.popleft(); level.append(n.val)
            if n.left:  q.append(n.left)
            if n.right: q.append(n.right)
        res.append(level)
    return res
```

## 9.3 Binary Search Tree (BST)

For every node: all left descendants < node < all right descendants. This gives O(log n) search/insert/delete *when balanced*, degrading to O(n) if it becomes a skewed chain.

```python
def bst_insert(root, val):
    if root is None:
        return TreeNode(val)
    if val < root.val:
        root.left = bst_insert(root.left, val)
    else:
        root.right = bst_insert(root.right, val)
    return root

def bst_search(root, val):
    while root and root.val != val:
        root = root.left if val < root.val else root.right
    return root
```

## 9.4 Self-balancing trees

To guarantee O(log n), balanced variants restructure on insert/delete: **AVL trees** (strictly balanced via rotations, faster lookups) and **Red-Black trees** (looser balance, faster inserts — used in many standard libraries/maps).

### Viva — Trees / BST

**Q What property defines a BST and what does inorder traversal give?**

**A** Left subtree < node < right subtree for every node. An inorder traversal of a BST visits nodes in ascending sorted order.

**Q Why can BST operations degrade to O(n)?**

**A** If insertions come in sorted order the tree becomes a skewed chain (essentially a linked list), so height = n. Self-balancing trees (AVL, Red-Black) prevent this by keeping height O(log n).

**Q AVL vs Red-Black tree?**

**A** AVL is more strictly balanced, giving faster lookups but more rotations on insert/delete. Red-Black is less strictly balanced, giving faster insertion/deletion; it's used where writes are frequent (e.g., library maps/sets).

**Q Difference between height and depth?**

**A** Depth is the distance (edges) from the root to a node; height is the distance from a node down to its deepest leaf. Tree height = height of the root.

**Q How do you do a level-order traversal?**

**A** Breadth-first using a queue: enqueue the root, then repeatedly dequeue a node, record it, and enqueue its children — processing one level at a time.

**Q How would you find the height of a binary tree?**

**A** Recursively: height = 1 + max(height(left), height(right)), with height of an empty node = 0 (or -1 counting edges). It's O(n).

# 10. Heaps & Priority Queues

*A heap is a complete binary tree satisfying the heap property; the standard implementation for priority queues.*

## 10.1 Types & properties

- **Min-heap** — parent ≤ children; root is the minimum.
- **Max-heap** — parent ≥ children; root is the maximum.
- Stored as an array: for index i, children are 2i+1 and 2i+2, parent is (i-1)//2.
- Insert & extract-min/max are O(log n); peek is O(1); building a heap from n items is O(n).

```python
import heapq

# Python heapq is a MIN-heap
h = []
heapq.heappush(h, 5)
heapq.heappush(h, 1)
heapq.heappush(h, 3)
print(heapq.heappop(h))    # 1 (smallest)

# k largest elements: O(n log k)
def k_largest(nums, k):
    return heapq.nlargest(k, nums)

# Max-heap trick: negate values
maxh = []
for x in [5,1,3]:
    heapq.heappush(maxh, -x)
largest = -heapq.heappop(maxh)   # 5
```

### Viva — Heaps

**Q What is a heap and how is it stored?**

**A** A complete binary tree where each parent respects the heap property (≤ or ≥ its children). It's stored compactly in an array using index arithmetic (children of i at 2i+1, 2i+2), so no pointers are needed.

**Q Complexity of heap operations?**

**A** Peek O(1); insert and extract-min/max O(log n) due to sift-up/sift-down; building a heap from n elements is O(n).

**Q How do you get a max-heap from Python's min-heap?**

**A** Push negated values (and negate again on pop), or store tuples with negated priorities. Python only provides a min-heap via heapq.

**Q Find the k largest elements efficiently.**

**A** Maintain a min-heap of size k while scanning: push each element and pop the smallest when size exceeds k. That's O(n log k) time and O(k) space — better than sorting when k ≪ n.

# 11. Tries (Prefix Trees)

*A trie stores strings by shared prefixes — excellent for autocomplete, spell-check and dictionary lookups.*

```python
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word):          # O(L), L = word length
        node = self.root
        for ch in word:
            node = node.children.setdefault(ch, TrieNode())
        node.is_end = True

    def search(self, word):          # O(L)
        node = self._find(word)
        return node is not None and node.is_end

    def starts_with(self, prefix):   # O(L)
        return self._find(prefix) is not None

    def _find(self, s):
        node = self.root
        for ch in s:
            if ch not in node.children:
                return None
            node = node.children[ch]
        return node
```

## Viva — Tries

**Q What problem does a trie solve better than a hash set?**

**A** Prefix queries. A trie finds all words with a given prefix (autocomplete) in O(prefix length), which a hash set can't do efficiently. Lookups are O(word length) regardless of how many words are stored.

**Q Trie space trade-off?**

**A** It can use more memory than a hash set because of per-character nodes, but shares common prefixes, saving space when many words overlap.

**Q Time complexity of insert/search in a trie?**

**A** O(L) where L is the length of the word — independent of the number of stored words.

# 12. Graphs & Traversals (BFS / DFS)

*A graph is a set of vertices connected by edges — models networks, maps, dependencies, social connections.*

## 12.1 Terminology & representations

- **Directed / undirected**; **weighted / unweighted**; **cyclic / acyclic**.
- **Adjacency list** — dict of vertex → neighbors; space O(V+E); best for sparse graphs.
- **Adjacency matrix** — V×V grid; O(1) edge check but O(V²) space; best for dense graphs.

## 12.2 BFS & DFS

```python
from collections import deque

def bfs(graph, start):
    """Level-by-level. Finds shortest path in UNWEIGHTED graphs. O(V+E)."""
    visited, order, q = {start}, [], deque([start])
    while q:
        node = q.popleft(); order.append(node)
        for nbr in graph[node]:
            if nbr not in visited:
                visited.add(nbr); q.append(nbr)
    return order

def dfs(graph, start, visited=None, order=None):
    """Goes deep first. Good for cycles, topological sort, components. O(V+E)."""
    if visited is None: visited, order = set(), []
    visited.add(start); order.append(start)
    for nbr in graph[start]:
        if nbr not in visited:
            dfs(graph, nbr, visited, order)
    return order
```

## 12.3 Topological sort (DAG)

Orders vertices so every edge u→v has u before v. Used for build/task scheduling and dependency resolution. Two methods: Kahn's algorithm (BFS with in-degrees) or DFS with a finish-time stack.

### Viva — Graphs

**Q BFS vs DFS — when do you use each?**

**A** BFS explores level by level (queue) and finds the shortest path in unweighted graphs; DFS goes deep (stack/recursion) and suits cycle detection, topological sorting, and finding connected components. Both are O(V+E).

**Q Adjacency list vs matrix?**

**A** List uses O(V+E) space and is efficient for sparse graphs; matrix uses $O(V^2)$ space but gives O(1) edge lookups, better for dense graphs.

**Q What is topological sort and when is it possible?**

**A** A linear ordering of vertices respecting all directed edges. It exists only for a DAG (directed acyclic graph); a cycle makes ordering impossible.

**Q How do you detect a cycle in a directed graph?**

**A** DFS while tracking nodes in the current recursion stack; if you reach a node already on the stack, there's a back edge → a cycle. In undirected graphs, a visited neighbor that isn't the parent indicates a cycle.

**Q How do you find connected components?**

**A** Run BFS/DFS from every unvisited vertex; each traversal marks one component. Count the number of traversals.

# 13. Shortest Path & MST Algorithms

## 13.1 Dijkstra's algorithm

Single-source shortest path for graphs with **non-negative** weights. Uses a min-heap to always expand the closest unvisited vertex. O((V+E) log V).

```python
import heapq
def dijkstra(graph, src):
    """graph: {u: [(v, weight), ...]}. Returns shortest distances from src."""
    dist = {u: float('inf') for u in graph}
    dist[src] = 0
    pq = [(0, src)]
    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]:
            continue
        for v, w in graph[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                heapq.heappush(pq, (dist[v], v))
    return dist
```

## 13.2 Other key algorithms

| Algorithm | Solves | Notes |
|---|---|---|
| Bellman-Ford | Shortest path with negative edges | Detects negative cycles; O(V·E) |
| Floyd-Warshall | All-pairs shortest paths | $O(V^3)$, DP based |
| Kruskal's | Minimum Spanning Tree | Sort edges + Union-Find |
| Prim's | Minimum Spanning Tree | Grow tree with a min-heap |

### Viva — Shortest Path / MST

**Q Why can't Dijkstra handle negative edges?**

**A** It commits to a vertex's shortest distance once it's popped from the heap, assuming no later path can be shorter. A negative edge could reduce a "finalized" distance, breaking that assumption — use Bellman-Ford instead.

**Q Dijkstra vs BFS for shortest path?**

**A** BFS finds shortest paths only in unweighted graphs (each edge = 1). Dijkstra handles non-negative weighted graphs using a priority queue.

**Q What is a Minimum Spanning Tree?**

**A** A subset of edges connecting all vertices with the minimum total weight and no cycles. Kruskal's (sort edges + union-find) and Prim's (grow from a vertex with a min-heap) both build one.

**Q What is Union-Find used for?**

**A** Efficiently tracking connected components and detecting cycles (Kruskal's MST). With path compression and union by rank it runs in near-constant amortized time per operation.

# 14. Searching Algorithms

## 14.1 Linear search

Scan every element; O(n). Works on unsorted data.

## 14.2 Binary search

Requires a **sorted** array. Repeatedly halve the search range; O(log n).

```python
def binary_search(arr, target):
    lo, hi = 0, len(arr) - 1
    while lo <= hi:
        mid = (lo + hi) // 2         # or lo + (hi-lo)//2 to avoid overflow
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1
```

**Binary search on the answer:** many "minimize the maximum / find smallest feasible X" problems are solved by binary-searching over the answer space with a feasibility check — a very common interview pattern.

### Viva — Searching

**Q Precondition for binary search?**

**A** The data must be sorted (or monotonic in the property you search on). Otherwise the halving logic is invalid.

**Q Why use lo + (hi-lo)//2 instead of (lo+hi)//2?**

**A** In languages with fixed-width integers, lo+hi can overflow; lo + (hi-lo)//2 avoids that. In Python integers are unbounded, so it's mainly a portability habit.

**Q Complexity of binary search and why?**

**A** O(log n) because each step discards half the remaining elements, so it takes about $\log_2 n$ steps to reach one element.

**Q What is "binary search on the answer"?**

**A** When the answer is monotonic (a feasibility check flips from false to true at some threshold), you binary-search the value range and test feasibility — turning an optimization into O(log range × check cost).

# 15. Sorting Algorithms

*Know each sort's mechanism, complexity, and stability — a guaranteed interview topic.*

| Algorithm | Best | Average | Worst | Space | Stable? |
|---|---|---|---|---|---|
| Bubble | O(n) | O(n²) | O(n²) | O(1) | Yes |
| Selection | O(n²) | O(n²) | O(n²) | O(1) | No |
| Insertion | O(n) | O(n²) | O(n²) | O(1) | Yes |
| Merge | O(n log n) | O(n log n) | O(n log n) | O(n) | Yes |
| Quick | O(n log n) | O(n log n) | O(n²) | O(log n) | No |
| Heap | O(n log n) | O(n log n) | O(n log n) | O(1) | No |

## 15.1 Merge sort (divide & conquer, stable)

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left  = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(a, b):
    res, i, j = [], 0, 0
    while i < len(a) and j < len(b):
        if a[i] <= b[j]:
            res.append(a[i]); i += 1
        else:
            res.append(b[j]); j += 1
    res.extend(a[i:]); res.extend(b[j:])
    return res
```

## 15.2 Quick sort (partition-based)

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr)//2]
    left  = [x for x in arr if x < pivot]
    mid   = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quick_sort(left) + mid + quick_sort(right)
```

## Viva — Sorting

**Q What does "stable sort" mean and when does it matter?**

**A** A stable sort preserves the relative order of equal keys. It matters when sorting by multiple criteria (e.g., sort by name, then stably by date keeps name order within each date).

**Q Merge sort vs quick sort?**

**A** Merge sort guarantees O(n log n) and is stable but needs O(n) extra space. Quicksort sorts in place with O(log n) stack and is usually faster in practice, but has $O(n^2)$ worst case with bad pivots (mitigated by randomized/median pivots).

**Q When is quicksort $O(n^2)$?**

**A** When pivots are consistently the smallest/largest element (e.g., already-sorted input with a first/last pivot), producing unbalanced partitions. Random or median-of-three pivots avoid this.

**Q Which sort does Python use?**

**A** Timsort — a hybrid of merge sort and insertion sort that exploits existing runs; stable and O(n log n) worst case, O(n) on nearly sorted data.

**Q Best sort for nearly-sorted data?**

**A** Insertion sort — O(n) when data is almost sorted because few shifts are needed. Timsort leverages this internally.

# 16. Divide & Conquer

*Break a problem into independent subproblems, solve recursively, then combine.*

Three steps: **Divide** the problem, **Conquer** subproblems recursively, **Combine** results. Examples: merge sort, quicksort, binary search, Karatsuba multiplication, closest pair of points.

> **Master Theorem** analyzes recurrences of the form T(n) = a·T(n/b) + f(n). For merge sort (a=2, b=2, f(n)=n): T(n) = O(n log n).

## Viva — Divide & Conquer

**Q What are the three steps of divide and conquer?**

**A** Divide the problem into subproblems, conquer them recursively, and combine their solutions into the final answer.

**Q Divide-and-conquer vs dynamic programming?**

**A** Both split into subproblems, but D&C subproblems are independent (no overlap), while DP subproblems overlap and are cached/reused. Merge sort is D&C; Fibonacci/knapsack is DP.

**Q What is the Master Theorem for?**

**A** A shortcut to solve divide-and-conquer recurrences T(n)=aT(n/b)+f(n) by comparing f(n) to n^(log_b a) to determine the asymptotic complexity.

# 17. Greedy Algorithms

*Make the locally optimal choice at each step, hoping it yields a global optimum. Works only when the problem has the greedy-choice property and optimal substructure.*

**Classic greedy problems:** activity selection, fractional knapsack, Huffman coding, Dijkstra, Kruskal's/Prim's MST, coin change (only for canonical coin systems).

```python
def activity_selection(activities):
    """activities: list of (start, end). Max non-overlapping activities."""
    activities.sort(key=lambda x: x[1])   # greedy: earliest finish first
    count, last_end = 0, float('-inf')
    for start, end in activities:
        if start >= last_end:
            count += 1
            last_end = end
    return count
```

## Viva — Greedy

**Q When does a greedy algorithm work?**

**A** When the problem has the greedy-choice property (a locally optimal choice leads to a global optimum) and optimal substructure. If not, greedy fails and you need DP or search.

**Q Give a case where greedy fails.**

**A** The 0/1 knapsack — taking the highest value/weight ratio greedily can be suboptimal because you can't take fractions; it needs dynamic programming. (Fractional knapsack, however, is solvable greedily.)

**Q Greedy vs dynamic programming?**

**A** Greedy makes one irrevocable choice per step and never reconsiders (fast, but only correct for certain problems). DP explores/combines overlapping subproblems and always finds the optimum where applicable, at higher cost.

**Q Is Dijkstra greedy?**

**A** Yes — it greedily finalizes the closest unvisited vertex at each step, which is provably correct for non-negative weights.

# 18. Dynamic Programming

*DP solves problems with overlapping subproblems and optimal substructure by storing subproblem results to avoid recomputation.*

## 18.1 Two styles

- **Top-down (memoization)** — recursion + cache. Natural, computes only needed subproblems.
- **Bottom-up (tabulation)** — iterative, fill a table from base cases up. Often more space/time efficient, no recursion overhead.

## 18.2 Worked example — 0/1 Knapsack

```python
def knapsack(weights, values, capacity):
    n = len(weights)
    # dp[i][w] = best value using first i items within weight w
    dp = [[0]*(capacity+1) for _ in range(n+1)]
    for i in range(1, n+1):
        for w in range(capacity+1):
            dp[i][w] = dp[i-1][w]                       # skip item i
            if weights[i-1] <= w:                       # or take it
                dp[i][w] = max(dp[i][w],
                               dp[i-1][w-weights[i-1]] + values[i-1])
    return dp[n][capacity]
```

## 18.3 Other classic DP problems

Fibonacci, Longest Common Subsequence, Longest Increasing Subsequence, Edit Distance, Coin Change, Matrix Chain Multiplication, subset sum, house robber, grid unique paths.

### Coin change (min coins)

```python
def coin_change(coins, amount):
    INF = float('inf')
    dp = [0] + [INF]*amount             # dp[a] = min coins for amount a
    for a in range(1, amount+1):
        for c in coins:
            if c <= a:
                dp[a] = min(dp[a], dp[a-c] + 1)
    return dp[amount] if dp[amount] != INF else -1
```

## Viva — Dynamic Programming

**Q What two properties make a problem suitable for DP?**

**A** Overlapping subproblems (the same subproblem recurs) and optimal substructure (the optimal solution is built from optimal solutions of subproblems).

**Q Memoization vs tabulation?**

**A** Memoization is top-down: recurse and cache results, computing only needed subproblems. Tabulation is bottom-up: iteratively fill a table from base cases. Tabulation avoids recursion overhead; memoization is easier to write from a recurrence.

**Q How do you approach a new DP problem?**

**A** Define the state (what parameters uniquely describe a subproblem), write the recurrence/transition, identify base cases, decide top-down vs bottom-up, and optionally optimize space by keeping only needed previous states.

**Q Why is naive recursive Fibonacci exponential and DP linear?**

**A** Naive recursion recomputes the same fib(k) exponentially many times (O(2^n)). DP caches each fib(k) once, so each of n subproblems is computed a single time — O(n).

**Q 0/1 knapsack complexity?**

**A** O(n × capacity) time and space (pseudo-polynomial, since it depends on the numeric capacity, not just n). Space can be reduced to O(capacity) using a rolling 1D array.

# 19. Final Mock Viva — Rapid Fire

**Q Which data structure for undo/redo?**

**A** Two stacks — one for undo, one for redo.

**Q Which for a browser's back button?**

**A** A stack (LIFO): push visited pages, pop to go back.

**Q Which for BFS?**

**A** A queue (FIFO).

**Q Which for task scheduling by priority?**

**A** A priority queue / heap.

**Q Which for autocomplete?**

**A** A trie (prefix tree).

**Q Fastest average lookup structure?**

**A** A hash table / dict — average O(1).

**Q Detecting a cycle in a linked list?**

**A** Floyd's tortoise and hare (slow/fast pointers), O(n) time O(1) space.

**Q Sorted-array search?**

**A** Binary search, O(log n).

**Q Guaranteed O(n log n), stable sort?**

**A** Merge sort.

**Q In-place, fast average sort?**

**A** Quicksort.

**Q Shortest path, non-negative weights?**

**A** Dijkstra's algorithm.

**Q Shortest path with negative weights?**

**A** Bellman-Ford (also detects negative cycles).

**Q All-pairs shortest path?**

**A** Floyd-Warshall, $O(V^3)$.

**Q Minimum spanning tree?**

**A** Kruskal's or Prim's.

**Q Dependency/build ordering?**

**A** Topological sort of a DAG.

**Q Overlapping subproblems + optimal substructure?**

**A** Dynamic programming.

**Q Which traversal gives sorted BST output?**

**A** Inorder.

**Q Balanced BST guaranteeing O(log n)?**

**A** AVL or Red-Black tree.

**How to answer any DSA question:** (1) restate & clarify, (2) give a brute-force baseline with its complexity, (3) propose an optimized approach and justify the data structure, (4) code it cleanly, (5) state final time/space complexity and edge cases. Communicating this process matters as much as the code.